Comparación Detallada: Rama `clean-dev` vs Rama `front`

Fecha de análisis: 22 de noviembre de 2025
Ramas comparadas:

- `clean-dev` (worktree en `/Users/pitu/Desktop/Entregas/MOA-clean-dev`)
- `front` (rama actual en `/Users/pitu/Desktop/Entregas/MOA`)

📊 Resumen Ejecutivo

Métrica	clean-dev	front	Diferencia
Total de archivos	235	455	+220 archivos
Archivos únicos	34	254	-
Archivos comunes	201	201	-
Archivos idénticos	45	45	22.4%
Archivos modificados	156	156	77.6%

Conclusión General

La rama `front` es **significativamente más completa** que `clean-dev`, con casi el doble de archivos. La rama `front` incluye:

- Sistema completo de testing (Jest)
- Funcionalidad de órdenes, direcciones, pagos y wishlist
- Módulos admin avanzados (dashboard, estadísticas)
- Sistema de seeders completo
- Optimizaciones de performance (virtualización, memoización)
- Múltiples utilidades y scripts

🆕 Archivos Únicos en `front` (254 archivos)

Backend - Testing & Scripts

Tests (`backend/__tests__/`)

- `adminOrderPermissions.test.js` - Tests de permisos de órdenes admin
- `adminOrderStatus.test.js` (+ duplicado con "2") - Tests de estados de órdenes
- `adminPermissions.test.js` (+ duplicado) - Tests de permisos admin

- `orderStock.test.js` (+ duplicado) - Tests de validación de stock
- `routes.test.js` (+ duplicado) - Tests de rutas
- `stockValidation.test.js` - Tests específicos de validación de stock
- `test-user.js` (+ duplicado) - Utilidades para testing

Scripts (`backend/scripts/`)

- `backup-db.sh` (+ duplicado "2") - Scripts de backup de base de datos
- `getAdminToken.js` (+ duplicado) - Obtener token de admin para testing
- `install-database.sh` - Instalación automatizada de DB
- `test-database.sql` - Scripts de prueba de DB
- `testLogin.mjs` (eliminado en versión actual)

Configuración Backend

- `backend/nodemon.json` (+ duplicado "2") - Config de auto-reload
- `backend/server.js` (+ duplicado "2") - Servidor principal
- `backend/index.js` - Entry point alternativo

Backend - Database

Schema

- `backend/database/schema/password_reset.sql` - Tabla de recuperación de contraseñas

Seeders Completos (`backend/database/seed/`)

- `addressesData.js` + `addressesSeed.js` - Datos y seeder de direcciones
- `cartsData.js` + `cartsSeed.js` - Datos y seeder de carritos
- `clientsData.js` + `clientsSeed.js` - Datos y seeder de clientes
- `ordersData.js` + `ordersSeed.js` - Datos y seeder de órdenes
- `wishlistData.js` + `wishlistsSeed.js` - Datos y seeder de wishlists

Backend - Routes & Controllers

Rutas Adicionales

- `backend/routes/addressRoutes.js` - CRUD de direcciones
- `backend/routes/configRoutes.js` - Configuración de tienda
- `backend/routes/healthRoutes.js` - Health checks

Controladores Adicionales

- `backend/src/controllers/addressController.js` - Gestión de direcciones
- `backend/src/controllers/adminController.js` - Panel admin general
- `backend/src/controllers/configController.js` - Configuración

- `backend/src/controllers/dashboardController.js` - Estadísticas y dashboard
- `backend/src/controllers/orderAdminController.js` - Admin de órdenes
- `backend/src/controllers/orderController.js` - Órdenes de usuario
- `backend/src/controllers/passwordResetController.js` - Recuperación de contraseña

Backend - Middleware & Models

Middleware

- `backend/src/middleware/dbHealthCheck.js` - Verificación de salud de DB
- `backend/src/middleware/validatePaymentMethod.js` - Validación de pagos
- `backend/src/middleware/verifyAdmin.js` - Verificación de rol admin

Modelos

- `backend/src/models/addressModel.js` - Modelo de direcciones
- `backend/src/models/configModel.js` - Modelo de configuración
- `backend/src/models/dashboardModel.js` - Modelo de estadísticas
- `backend/src/models/orderAdminModel.js` - Modelo admin de órdenes
- `backend/src/models/orderModel.js` - Modelo de órdenes
- `backend/src/models/passwordResetModel.js` - Modelo de recuperación

Frontend - Módulo Órdenes Completo

Nuevo módulo: `frontend/src/modules/orders/`

- `hooks/useCheckout.js` - Hook de proceso de compra
- `hooks/useOrderDetail.js` - Detalle de orden
- `hooks/useOrderSummary.js` - Resumen de orden
- `pages/CheckoutPage.jsx` - Página de checkout
- `pages/OrderConfirmationPage.jsx` - Confirmación de compra
- `pages/OrderDetailPage.jsx` - Detalle de orden

Frontend - Módulo Wishlist

Nuevo módulo: `frontend/src/modules/wishlist/`

- `components/WishlistProductCard.jsx` - Card de producto en wishlist
- `hooks/useWishlistManager.js` - Gestión de wishlist
- `pages/WishlistPageV2.jsx` - Página de wishlist mejorada
- `WishlistContext.jsx` - Context de wishlist

Frontend - Admin Dashboard Avanzado

Componentes Dashboard (`frontend/src/modules/admin/components/dashboard/`)

- `AdminAnalyticsSection.jsx` - Sección de analíticas
- `AdminChart.jsx` - Gráficos
- `AdminChartsSection.jsx` - Sección de gráficos
- `AdminDashboardCard.jsx` - Cards del dashboard
- `AdminDataCards.jsx` - Cards de datos
- `AdminHeaderStats.jsx` - Estadísticas del header
- `AdminKPICard.jsx` - Cards de KPIs
- `AdminQuickActions.jsx` - Acciones rápidas
- `AdminStatsSection.jsx` - Sección de estadísticas
- `DashboardHeader.jsx` - Header del dashboard
- `OrdersChart.jsx` - Gráfico de órdenes
- `RecentOrders.jsx` - Órdenes recientes
- `RevenueChart.jsx` - Gráfico de ingresos
- `SalesChart.jsx` - Gráfico de ventas
- `SalesOverview.jsx` - Vista general de ventas
- `TopProducts.jsx` - Productos más vendidos

Páginas Admin Adicionales

- `AdminCategoriesPage.jsx` - Gestión de categorías
- `AdminCustomersPage.jsx` - Gestión de clientes
- `AdminDashboardPage.jsx` - Dashboard principal
- `AdminOrderDetailPage.jsx` - Detalle de orden admin
- `AdminOrdersPageV2.jsx` - Gestión de órdenes v2
- `AdminPage.jsx` - Layout admin
- `AdminProductsPageV2.jsx` - Gestión de productos v2

Componentes Admin

- `AdminPageHeader.jsx` - Header de páginas admin
- `ExportButton.jsx` - Botón de exportación
- `OrderStatusBadge.jsx` - Badge de estados
- `ResponsiveRowActions.jsx` - Acciones responsive
- `VirtualizedTable.jsx` - Tabla virtualizada

Frontend - Hooks Adicionales

Performance & Optimización

- `hooks/useDebounce.js` - Debouncing
- `hooks/useDebouncedValue.js` - Valor con debounce
- `hooks/useOnClickOutside.js` - Click fuera de elemento
- `hooks/usePrefetch.js` - Prefetching de datos
- `hooks/useQueryWithFallback.js` - Query con fallback

Utilidades

- `hooks/useCurrentUser.js` - Usuario actual
- `hooks/useErrorHandler.js` - Manejo de errores
- `hooks/useFilteredData.js` - Filtrado de datos
- `hooks/useLocalStorage.js` - LocalStorage
- `hooks/useMediaQuery.js` - Media queries
- `hooks/useOrderStatusTransitions.js` - Transiciones de estados
- `hooks/useRegionesYComunas.js` - Regiones y comunas de Chile
- `hooks/useToggle.js` - Toggle state

Frontend - Componentes UI Avanzados

Shadcn/UI Components

- Múltiples componentes adicionales de Radix UI
- Sistema de diseño completo

Componentes Propios

- `ErrorBoundary.jsx` - Boundary de errores
- `LoadingBar.jsx` - Barra de carga
- `LoadingSpinner.jsx` - Spinner de carga
- `PriceCard.jsx` - Card de precio
- `ProductCardSkeleton.jsx` - Skeleton de producto
- `SearchBar.jsx` - Barra de búsqueda
- `SuspenseErrorBoundary.jsx` - Boundary con Suspense

Frontend - Utils & Services

Servicios API

- `services/addresses.api.js` - API de direcciones
- `services/cart.api.js` - API de carrito
- `services/categories.api.js` - API de categorías
- `services/checkout.api.js` - API de checkout
- `services/config.api.js` - API de configuración
- `services/dashboard.api.js` - API de dashboard
- `services/orders-admin.api.js` - API de órdenes admin
- `services/users.api.js` - API de usuarios
- `services/wishlist.api.js` - API de wishlist

Utilities

- `utils/fileUtils.js` - Utilidades de archivos

- `utils/formatters.js` - Formateadores
- `utils/queryClient.js` - Cliente de queries
- `utils/sort.js` - Utilidades de ordenamiento
- `utils/test-utils.jsx` - Utilidades de testing

Frontend - Configuración & Contexto

Contexts

- `context/AuthOptimized.jsx` - Auth optimizado
- `context/CartOptimized.jsx` - Cart optimizado

Config

- `config/api-paths.js` - Paths de API centralizados
- `config/order-status-config.js` - Config de estados de orden
- `config/site-config.js` - Configuración del sitio

Frontend - Scripts

- `scripts/analyze-bundle.js` - Análisis de bundle
- `scripts/check-routes.js` - Verificación de rutas
- `scripts/delete-modules.js` - Limpieza de módulos
- `scripts/extract-shadcn-from-ui.js` - Extracción de componentes
- `scripts/report-duplicates.js` - Reporte de duplicados
- `scripts/test-auth-mock.js` - Test de auth mock

Documentación Adicional

Más de **20 archivos de documentación** adicionales en `docs/misDOCS/`, incluyendo:

- Guías de optimización
- Documentación de flujos
- Manuales de usuario
- Especificaciones de API
- Auditorías de código

Configuración Global

- `.github/copilot-instructions.md` - Instrucciones de Copilot
- `.vscode/settings.json` - Configuración de VSCode
- Múltiples archivos de configuración duplicados (marcados con "2")

🎯 Archivos Únicos en `clean-dev` (34 archivos)

Backend

Database

- `database/schema/DML.sql` - Data Manipulation Language (queries de ejemplo)

Controllers & Models (Nomenclatura diferente)

- `src/controllers/ordersController.js` - En `front` se llama `orderController.js` y `orderAdminController.js`
- `src/models/ordersModel.js` - En `front` está dividido en varios modelos

Frontend - Mocks (Sistema de datos mock)

API Mocks (`frontend/src/mocks/api/`)

- `home.js` - Mock de API home
- `index.js` - Exportador de mocks
- `products.js` - Mock de API productos

Database Mocks (`frontend/src/mocks/database/`)

- `carts.js` - Datos mock de carritos
- `categories.js` - Datos mock de categorías
- `collections.js` - Datos mock de colecciones
- `content.js` - Contenido mock
- `customers.js` - Clientes mock
- `index.js` - Exportador de mocks
- `orders.js` - Órdenes mock
- `products.js` - Productos mock
- `site.js` - Configuración de sitio mock
- `users.js` - Usuarios mock

Frontend - Admin (Versiones antiguas/alternativas)

Componentes

- `modules/admin/components/AdminStatsCard.jsx` - En `front` está en `dashboard/`
- `modules/admin/components/ProductFilters.jsx` - En `front` es parte de productos

Páginas Admin Antiguas

- `modules/admin/pages/AdminCollectionsPage.jsx` - No existe en `front`
- `modules/admin/pages/AdminDashboard.jsx` - En `front` es `AdminDashboardPage.jsx`
- `modules/admin/pages/AdminSettingsPage.jsx` - No implementado en `front`
- `modules/admin/pages/AdminTestPage.jsx` - Página de testing

Páginas de Órdenes (estructura antigua)

- `modules/admin/pages/orders/OrdersDetailPage.jsx` - En `front` es `AdminOrderDetailPage.jsx`
- `modules/admin/pages/orders/OrdersListPage.jsx` - Reemplazado por `AdminOrdersPageV2.jsx`
- `modules/admin/pages/orders/OrdersPage.jsx` - Layout antiguo

Páginas de Productos (estructura antigua)

- `modules/admin/pages/products/NewProductPage.jsx` - No existe en `front`
- `modules/admin/pages/products/ProductsAdminPage.jsx` - En `front` es `AdminProductsPageV2.jsx`

Frontend - Hooks

- `modules/profile/hooks/useMyOrders.js` - En `front` está en `modules/orders/hooks/`

Configuración

- `.github/chatmodes/moa.chatmode.md` - Modo de chat personalizado

Documentación

- `docs/assets.md` - Documentación de assets
- `docs/DEPENDENCIAS.md` - Lista de dependencias
- `docs/ENDPOINTS_Y_MIGRACION.md` - Endpoints y migración
- `docs/STATUS.md` - Estado del proyecto

🔄 Archivos Comunes pero Diferentes (156 archivos)

Backend - Diferencias Principales

`package.json`

clean-dev:

- Entry point: `index.js`
- Scripts básicos: `dev`, `start`, seeders simples

front:

- Entry point: `server.js`
- Scripts avanzados: testing (Jest), múltiples seeders, database management
- Dependencias adicionales:
  - `compression` - Compresión de respuestas
  - `express-rate-limit` - Rate limiting
  - `nodemailer` - Envío de emails

    - `stripe` - Pagos
- DevDependencies: Babel, Jest configurado

`database/config.js`

Diferencias en configuración de pool de conexiones y variables de entorno.

`database/schema/DDL.sql`

front tiene un schema **significativamente más completo**:

- Tabla `password_reset` para recuperación de contraseñas
- Índices adicionales para performance
- Constraints mejorados
- Triggers y funciones adicionales

Routes (`backend/routes/*.js`)

front tiene rutas más completas con:

- Validación de datos mejorada
- Rate limiting en rutas sensibles
- Middleware de autorización más granular
- Endpoints adicionales (admin, config, health)

Controllers

front tiene controladores más robustos con:

- Manejo de errores centralizado
- Validaciones más estrictas
- Soporte para paginación y filtros avanzados
- Logs estructurados

Models

front usa el patrón de "lanzar errores" consistentemente, mientras que **clean-dev** tiene patrones mixtos.

Middleware

front incluye:

- `tokenMiddleware.js` - JWT mejorado con refresh tokens
- `verifyAdmin.js` - Verificación de roles
- Middleware de validación adicional

Frontend - Diferencias Principales

`package.json`

clean-dev:

- Dependencias básicas
- Scripts simples

front:

- Dependencias avanzadas:
  - `@tanstack/react-virtual` - Virtualización de listas
  - `class-variance-authority` - Variantes de componentes
  - `date-fns` - Manejo de fechas
  - `framer-motion` - Animaciones
  - `jwt-decode` - Decodificación de JWT
  - `react-countup` - Animaciones de números
  - `recharts` - Gráficos
  - `xlsx` - Exportación a Excel
- Scripts de testing y análisis

`vite.config.js`

front tiene configuración avanzada:

- Code splitting manual
- Chunks optimizados (vendor, react-query, ui)
- Alias de imports (`@/`)
- Optimizaciones de build

`tailwind.config.js`

front incluye:

- Tema personalizado completo
- Colores extendidos
- Animaciones personalizadas
- Configuración de plugins (typography, forms)

Módulos (`frontend/src/modules/`)

Admin

front tiene una arquitectura completamente renovada:

- Dashboard con gráficos y KPIs
- Tablas virtualizadas para performance
- Exportación de datos

- Gestión avanzada de órdenes y clientes

Auth

front incluye:

- Recuperación de contraseñas
- Protección de rutas mejorada
- Manejo de refresh tokens

Cart

front tiene:

- Context optimizado
- Sincronización con backend
- Validación de stock en tiempo real

Products

front incluye:

- Filtros avanzados (precio, categoría, búsqueda)
- Prefetching en hover
- Virtualización de listas largas
- Galería de imágenes mejorada

Profile

front tiene:

- Sección de órdenes
- Gestión de direcciones
- Wishlist integrada
- Información de usuario editable

Componentes (`frontend/src/components/`)

front tiene un sistema de componentes mucho más completo:

UI Components

- Sistema de diseño consistente
- Componentes de Radix UI integrados
- Variantes con CVA
- Componentes memoizados para performance

Data Display

- `DataTableV2.jsx` - Tabla con virtualización
- `TableToolbar.jsx` - Barra de herramientas avanzada
- `VirtualizedTable.jsx` - Tabla virtualizada

Layout

- `Navbar.jsx` - Navegación con búsqueda y cart
- `Footer.jsx` - Footer completo
- Error boundaries

Hooks (`frontend/src/hooks/`)

front tiene **más de 15 hooks personalizados** vs prácticamente ninguno en **clean-dev**.

Services (`frontend/src/services/`)

front tiene una arquitectura de servicios completamente estructurada:

- `api-client.js` - Cliente HTTP centralizado con interceptors
- 10+ servicios API específicos por módulo
- Manejo de errores centralizado
- Retry logic

Utils (`frontend/src/utils/`)

front incluye múltiples utilidades:

- `cn.js` - Merge de clases (clsx + tailwind-merge)
- `formatters.js` - Formateo de moneda, fechas, etc.
- `normalizers.js` - Normalización de datos
- `sort.js` - Utilidades de ordenamiento
- `date.js` - Manejo de fechas

Routes & Navigation

front tiene:

- Lazy loading de todas las páginas
- Code splitting por módulo
- Protected routes con verificación de roles
- Rutas de admin separadas

🎨 Características Únicas por Rama

`clean-dev` es más...

- ✅ **Minimalista** - Estructura básica sin features avanzadas
- ✅ **Ligero** - Menos dependencias (ideal para desarrollo inicial)
- ✅ **Simple** - Código más directo y fácil de seguir
- ✅ **Con Mocks** - Sistema completo de datos mock para desarrollo sin backend

`front` es más...

- ✅ **Completo** - Todas las features del e-commerce implementadas
- ✅ **Optimizado** - Code splitting, memoización, virtualización
- ✅ **Robusto** - Testing, error handling, validaciones
- ✅ **Escalable** - Arquitectura preparada para crecimiento
- ✅ **Profesional** - Documentación, scripts, herramientas de desarrollo
- ✅ **Listo para Producción** - Rate limiting, compresión, health checks

---

📈 Diferencias en Arquitectura

Backend

Aspecto	clean-dev	front
Entry Point	`index.js`	`server.js`
Testing	❌ No configurado	✅ Jest + 7 test suites
Database Seeders	3 básicos	8 completos
Controllers	5 básicos	13 completos
Models	7 básicos	10 completos
Routes	8 básicas	12 completas
Middleware	2 básicos	6 completos
Error Handling	Básico	Centralizado y robusto
Security	Básico	Rate limiting, CORS avanzado
Email	❌ No	✅ Nodemailer
Payments	❌ No	✅ Stripe

Frontend

Aspecto	clean-dev	front
Módulos	8 módulos	11 módulos

Aspecto	clean-dev	front
Páginas	~15 páginas	~40 páginas
Componentes UI	~20	~80+
Hooks Personalizados	~5	~20
Servicios API	3 básicos	12 completos
Utils	3 básicos	10+ completos
State Management	Context básico	Context + TanStack Query
Code Splitting	❌ No	✅ Optimizado
Virtualización	❌ No	✅ Tablas y listas
Testing	❌ No	✅ Config de tests
Error Boundaries	❌ No	✅ Múltiples niveles
Mocks	✅ Sistema completo	❌ Removidos
Performance	Básico	Optimizado (memoización, prefetch)

🔍 Diferencias Clave en Funcionalidad

Features Solo en `front`

Backend

1. **Sistema de Órdenes Completo**
   - CRUD de órdenes
   - Estados de orden con transiciones
   - Admin de órdenes
   - Historial de cambios
2. **Sistema de Direcciones**
   - CRUD de direcciones
   - Dirección por defecto
   - Validación de regiones
3. **Sistema de Pagos**
   - Integración con Stripe
   - Validación de métodos de pago
   - Historial de transacciones

4. **Dashboard Admin**
    - Estadísticas de ventas
    - KPIs
    - Gráficos
    - Top productos
5. **Configuración de Tienda**
    - Configuración global
    - Health checks
    - Logs estructurados
6. **Recuperación de Contraseñas**
    - Reset de contraseña por email
    - Tokens de recuperación
7. **Testing Completo**
    - 7 test suites
    - Tests de permisos
    - Tests de validación de stock
    - Tests de rutas

Frontend

1. **Módulo de Órdenes**
    - Proceso de checkout completo
    - Confirmación de compra
    - Historial de órdenes
    - Detalle de orden
2. **Dashboard Admin Avanzado**
    - Gráficos interactivos (Recharts)
    - KPIs animados
    - Tablas virtualizadas
    - Exportación a Excel
3. **Gestión de Clientes**
    - Listado de clientes
    - Detalle de cliente
    - Historial de compras
4. **Gestión de Categorías**

    - CRUD completo
    - Drag & drop
    - Imágenes

5. **Wishlist**
    - Agregar/remover productos
    - Compartir wishlist
    - Sincronización con backend

6. **Optimizaciones de Performance**
    - Code splitting
    - Lazy loading
    - Prefetching
    - Virtualización
    - Memoización

7. **Error Handling Robusto**
    - Error boundaries
    - Suspense boundaries
    - Manejo de errores por módulo

Features Solo en `clean-dev`

1. **Sistema de Mocks Completo**
    - API mocks
    - Database mocks
    - Desarrollo sin backend

2. **Páginas Admin Alternativas**
    - `AdminTestPage` - Testing interno
    - `AdminSettingsPage` - Configuración (no implementado en front)
    - `AdminCollectionsPage` - Colecciones (no existe en front)

🚀 Recomendaciones

Si estás en `clean-dev` y quieres las features de `front`:

1. **Prioridad Alta - Features Críticas:**
    - ✅ Sistema de órdenes completo (backend + frontend)
    - ✅ Sistema de direcciones
    - ✅ Checkout flow

    - ✅ Testing (Jest)
    - ✅ Error handling robusto

2. **Prioridad Media - Mejoras de UX:**
    - ✅ Dashboard admin con gráficos
    - ✅ Tablas virtualizadas
    - ✅ Wishlist
    - ✅ Recuperación de contraseñas
    - ✅ Optimizaciones de performance

3. **Prioridad Baja - Nice to Have:**
    - ✅ Exportación a Excel
    - ✅ Scripts de desarrollo
    - ✅ Documentación extendida

Si estás en `front` y quieres algo de `clean-dev`:

1. **Considerar agregar:**
    - ✅ Sistema de mocks para desarrollo sin backend
    - ✅ `AdminSettingsPage` (configuración de tienda en UI)
    - ✅ `AdminCollectionsPage` (si planeas agregar colecciones)

📝 Archivos con Diferencias Significativas

Backend

1. **`package.json`**
    - `front` tiene 6 dependencias adicionales
    - 10+ scripts adicionales en `front`

2. **`database/schema/DDL.sql`**
    - `front` tiene tabla adicional: `password_reset`
    - Índices y constraints mejorados

3. **`routes/*.js` (todos diferentes)**
    - `front` tiene validaciones y middleware adicional

4. **`controllers/*.js` (mayoría diferentes)**
    - `front` usa patrón de errores consistente
    - Mejor separación de responsabilidades

5. **`models/*.js` (mayoría diferentes)**

    - `front` tiene modelos más específicos y separados

Frontend

1. **`package.json`**
    - `front` tiene 12 dependencias adicionales
    - Scripts de testing y análisis
2. **`vite.config.js`**
    - `front` tiene configuración avanzada de chunks
3. **`tailwind.config.js`**
    - `front` tiene tema personalizado completo
4. **`src/app/App.jsx`**
    - `front` tiene lazy loading y error boundaries
5. **`src/modules/*/` (casi todos diferentes)**
    - Arquitectura más madura en `front`
6. **`src/components/` (mayoría diferentes)**
    - Sistema de componentes más completo en `front`
7. **`src/services/` (todos diferentes)**
    - Cliente HTTP centralizado en `front`
8. **`src/hooks/` (mayoría únicos en front)**
    - 15+ hooks personalizados en `front`

🎯 Conclusión Final

`clean-dev`

Es ideal para: Empezar un proyecto desde cero, entender la arquitectura básica, desarrollo con mocks.

Fortalezas:

- Código simple y fácil de entender
- Sistema de mocks completo
- Estructura clara

Debilidades:

- No listo para producción
- Features incompletas
- Sin testing
- Sin optimizaciones

`front`

Es ideal para: Proyecto en producción o cerca de producción, e-commerce completo.

Fortalezas:

- Todas las features implementadas
- Testing robusto
- Optimizado para performance
- Documentación completa
- Listo para producción

Debilidades:

- Más complejo
- Curva de aprendizaje mayor
- No tiene sistema de mocks (requiere backend real)

Recomendación General

Si tu objetivo es:

- **Aprender:** Usa `clean-dev` como base y ve agregando features
- **Producción:** Usa `front` directamente
- **Desarrollo rápido:** Usa `clean-dev` con mocks
- **E-commerce completo:** Definitivamente `front`

Next Steps Sugeridos

1. **Merge Strategy:** Considerar hacer merge de features específicas de `clean-dev` a `front` (principalmente el sistema de mocks)
2. **Documentation:** Mantener esta documentación actualizada
3. **Testing:** Agregar más tests en `front` basándose en la estructura de `clean-dev`
4. **Refactor:** Eliminar archivos duplicados (marcados con "2") en `front`

Nota: Este análisis se basa en 455 archivos en `front` y 235 en `clean-dev`, con 156 archivos compartidos que tienen diferencias significativas.